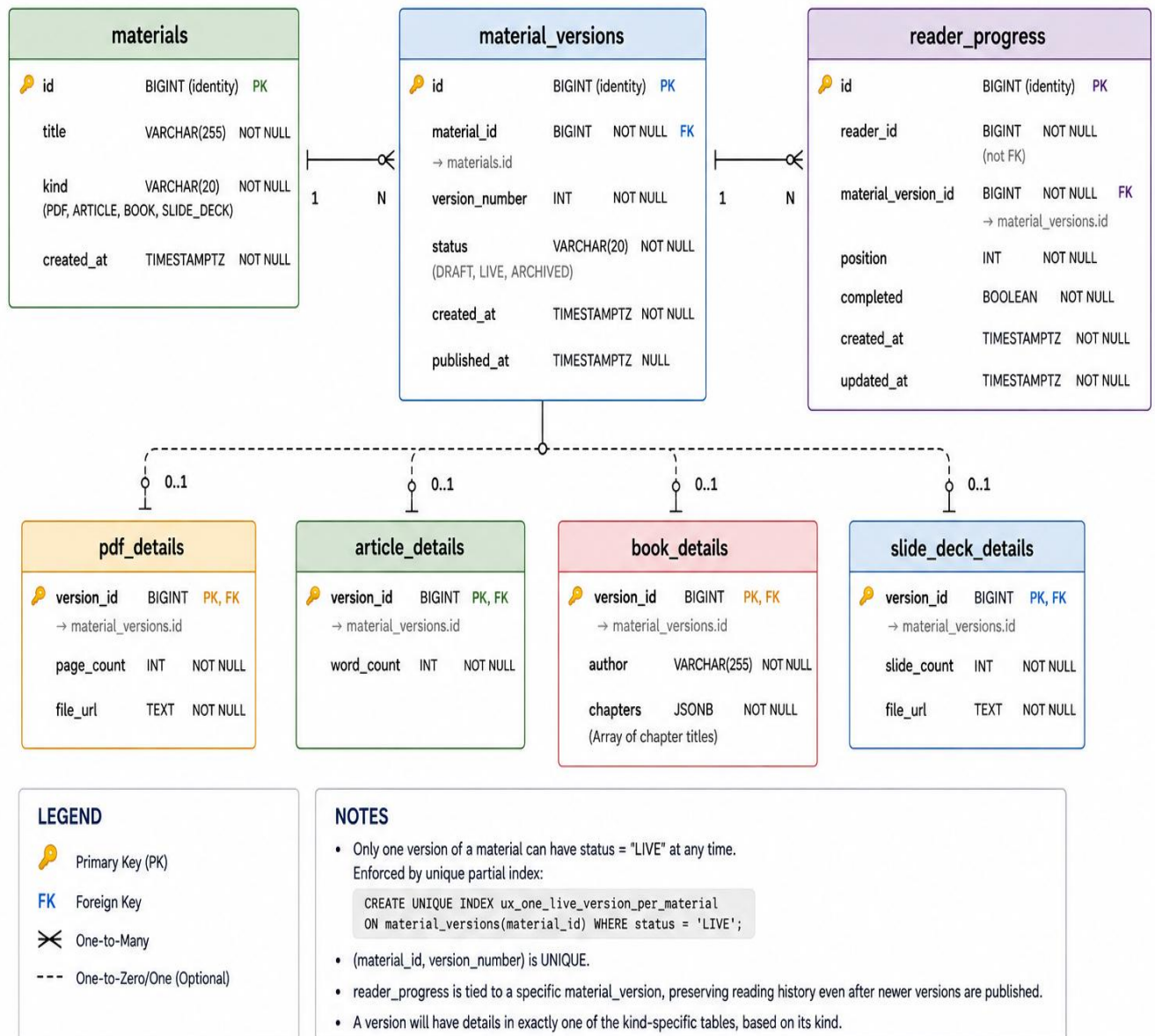


Reading Material Library — Design Document

Data model

The design separates the logical material, its immutable versions, kind-specific details, and reader progress.



Kind-specific details are stored in separate tables because each kind has different real fields and validation rules. A PDF's `page_count` is therefore a normal database column, while a book's actual chapter list is stored as JSONB. This allows database-side queries such as "PDFs with at least 100 pages" without loading all materials into Java.

The database also has a unique constraint on (material_id, version_number) and a partial unique index:

```
CREATE UNIQUE INDEX ux_one_live_version_per_material
ON material_versions(material_id)
WHERE status = 'LIVE';
```

This makes it impossible for the database to contain two LIVE versions for the same material.

Adding a fifth kind

A new kind, for example VIDEO, would not require changing *materials*, *material_versions*, *reader_progress*, or existing rows

I would add:

video_details -> version_id, duration_minutes

and add the corresponding Java validation, creation, copy, update, and reader-position logic.

The cost is additional code and a new detail table, but existing tables and data remain unchanged. This was chosen because the assignment explicitly requires future kinds without modifying existing tables or rows.

Publishing and concurrency.

An admin never edits a LIVE version. To change material, the service copies the current LIVE version into a new DRAFT, edits the DRAFT, and publishes it.

Publishing is transactional and locks the material row using PESSIMISTIC_WRITE. The service first changes the existing LIVE version to ARCHIVED and flushes that change, then changes the target version to LIVE and flushes again.

The row lock serializes concurrent publication attempts for the same material. The PostgreSQL partial unique index is the final database-level guarantee that two LIVE versions can never exist at the same time.

Older versions are never deleted, so publishing an older ARCHIVED version provides rollback.

Reader position

Reader progress is stored against material_version_id, not only material_id.

The meaning of position depends on the kind:

PDF → page number

Article → percentage

Book → *chapter number*

Slide deck → *slide number*

Completion is calculated by the service:

PDF → *last page reached*

Article → *100%*

Book → *last chapter reached*

Slide deck → *last slide reached*

The reader cannot supply the completed value.

If a reader is on version 1 at page 20 and version 2 is later published, the progress remains attached to version 1. The normal open endpoint returns the current LIVE version, while the continue-reading endpoint returns the reader's saved version and position.

What happens when a version stops being LIVE?

A reader's saved progress remains attached to the old version.

When the reader opens the material normally, they see the new LIVE version. Their previous progress is not silently copied to a different version because page, chapter, slide, or percentage positions may no longer mean the same thing after an edit.

The trade-off is that a reader may need to start progress on the new version again. This is deliberate because automatically mapping positions between different versions could silently create incorrect reading positions.

Historical progress remains available through the reader's own list and continue-reading flow.

What happens with 50,000 materials and 100,000 readers?

The design avoids loading the full library into Java.

Catalog searches and pagination are database-backed. The PDF page-count search is performed directly in PostgreSQL. Reader lists are also paginated in the database and use indexes on the relevant access paths.

Important indexes include:

UNIQUE(material_id, version_number)

partial unique index on LIVE versions

materials(kind)

pdf_details(page_count)

reader_progress(reader_id, completed, updated_at)

UNIQUE(reader_id, material_version_id)

Reader progress is isolated by **reader ID and version ID**, and duplicate progress rows for the same reader/version are prevented by the database.

For a real production deployment, I would additionally consider connection-pool sizing, read replicas if read traffic becomes dominant, caching for frequently accessed catalog data, observability, backups, and load testing before increasing traffic substantially.

What is not finished / what I would change before real users

The assignment intentionally excludes authentication, so X-Reader-Id is currently trusted input. Production would replace this with authenticated user identity and authorization.

I would also add stronger API authentication/authorization, structured application logging and metrics, centralized tracing, rate limiting where appropriate, backup/restore procedures, CI/CD, security hardening, and load testing.

The current service is designed and tested as a production-oriented take-home implementation, but those operational and security capabilities would be required before exposing it to real users.